Java Generics

Java is typed language.
Type Safety→ example: we can't store string type value directly into integer datatype.

Upcasting & Downcasting

- **These are done in non-primitive.**

UpCasting→ specific class ko general class me convert kr dena, matlab sub-class ko super-class ke reference me Assign krna.

- Ye casting automatically ho jati ha, no casting keyword required.

```
    String s = "Hello";
        Object obj = s;
            System.out.println(obj);// Hello.
```

DownCasting→To convert super-class into the sub-class, keyword of sub-class required.

```
Object obj1 = "Priyansh";
        String s1 = (String) obj1;
            System.out.println(s1);// Priyansh
```

DownCasting exception→ Run-time exception:

```
Object obj2 = 19;
        String s2 = (String) obj2;
            System.out.println(s2);// ClassCastException--> Run-time exception
```

Problem before Generic→

- hume ya to har datatype **instance variable** k lia alag-alag class bnani pdegi ya fir same **class** me har type ke **instance variable** ke liya alag-alag **getters, settes, constructors** bnane pdege.
- Ya fir **object type** ka instance variable bnana pdega, pr usme **type information** lost ho jati ha, because it is **too generic** class.
- Or **object type instance variables** me hum har datatype ke liye main funtion me downcasting krni pdegi, or ye bhot **risky** ho jata ha, therfore using Object and make it too generic is not good & safe way.

```
public class ProblemBeforeGeneric {
    public static void main(String[] args) {
        Box b1 = new Box (10);
        Box b2 = new Box ("Hello");
        Box b3 = new Box (true);
            System.out.println(b1.getValue());// ans->10
            //System.out.println(b1.getValue() + 5);// error--> not sure.
                        // so we have to do--> DownCasting..
        Integer x = (Integer) b1.getValue();
        String s = (String) b2.getValue();
        Boolean b = (Boolean) b3.getValue();
            System.out.println(x + 5);
            System.out.println(s + 5);
            System.out.println(b);
    }
}
class Box {
    private Object value;
        Box(Object value) {
            this.value = value;
                //System.out.println(this.value);// constructor dalte hi print ho jaye.
        }
        public Object getValue(){
                //System.out.println(this.value);// main function me print statement na likhni pde.
            return this.value;
```

```
        }
        public void setValue(Object value){
            this.value = value;
        }
}
```

GENERIC CLASS→ye kisi bhi class ko kisi bhi type me kaam krne k lia access delati ha.

- **<T> ye parameter ha jiski madad se hm kisi bhi type m kaam kr skte ha.**
- **<T> ye methode me bhi same kaam krta ha.**

```
        Pair <Integer, String> p1 = new Pair<>(14, "Ram");
            System.out.println(p1.first + ", " + p1.second);
    }
}
class Pair < T, U > {
    T first;
    U second;
        Pair(T first, U second){
            this.first = first;
            this.second = second;
          }
```

Generic method→

```
public class GenericMethods {
    public static void main(String[] args) {
        Integer y = getResult(5);
            System.out.println(y);
        String s = getResult("Priyansh");
            System.out.println(s);
        printPair(true, "Priyanshu"); // here is type inference.
    }
    public static <T> T getResult(T v){    //<T> TYPE PARAMETER.
        return v;
    }
    public static <T, U> void printPair(T first, U second){
        System.out.println(first + ", " + second);
    }
}
```

How to set Restrictions in generic classes→

```
public class RestrictedGeneric {
    public static void main(String[] args) {

        Box <Integer> b1 = new Box();
            b1.value = 5;
            b1.printDouble();
    }
}
            //Bounds in generics-->
                // Upper bound.
class Box <T extends Number> {// 'Number' is parent class of all numerical classes,yaha pr  hm ye bhi kr skte ha-->
//        <T extends Number  & interface1, interface2, etc>{}
    T value;
        public void printDouble(){
            System.out.println(value.doubleValue());
        }
}
// Agar me chahta hu ki meri class bas har tarah ke numbers ke sath kaam kr ske
//  or 'string', 'boolean', etc ko restrict kr de.
```